LLVM-Based Efficient Hybrid Cache and TCM Memory Allocation for Low-Latency

Gihyeon Jeon and Daejin Park, *Member, IEEE,*

***Abstract*—Cache memory has been introduced to accelerate embedded system performance and is automatically managed without programmer intervention through hardware-based cache controllers. However, since it operates based on typical variable access patterns, opportunities for complementary approaches remain. In contrast, Scratchpad Memory (SPM) or Tightly Coupled Memory (TCM), which is closely coupled with the processor core, require direct programmer control over data placement or compiler optimization support. Nevertheless, this control enables deterministic execution as intended by the programmer, allowing TCM to compensate for the shortcomings of cache memory. While various memory system optimization studies have been conducted, research that enhances applicability by utilizing LLVM passes for LLVM compiler optimization development or preserving existing build systems has been limited. This paper proposes a new method that extracts metadata via an LLVM pass seamlessly integrated with existing build systems and uses it to allocate data to DTCM RAM in a tiered manner. Experiments using CoreMark-Pro, MiBench, and STREAM benchmarks compared performance with access frequency-based dynamic programming allocation. Results showed the locality-based method achieved more stable performance and reduced execution cycles by 1.4% to 8%, demonstrating that locality-aware allocation is more effective than frequency-only approaches in cache-TCM hybrid environments.**

***Index Terms*—LLVM pass, profiling, optimization, TCM, cache**

I. Introduction

In modern embedded systems, memory system optimization has emerged as a key strategy for performance acceleration. Two prominent approaches include cache memory and tightly coupled memory (TCM). Cache memory provides the advantage of automatic system acceleration without programmer intervention through dedicated cache controllers. However, since it operates based on typical variable access patterns, it cannot optimize the placement of all variables, and suffers from significant area overhead and high energy consumption.

In contrast, TCM can compensate for these shortcomings. TCM is a software-managed memory directly connected to the core, allows for deterministic single-cycle access, and offers the advantages of a smaller area footprint and higher power efficiency due to the absence of tag memory and controllers [1], [2]. These characteristics make it an ideal candidate to assist in scenarios where the cache falls short, particularly for data with irregular or cache-unfriendly access patterns. The challenge, however, lies in its explicit management: the programmer or compiler must intelligently decide which data to place in the TCM to maximize performance.

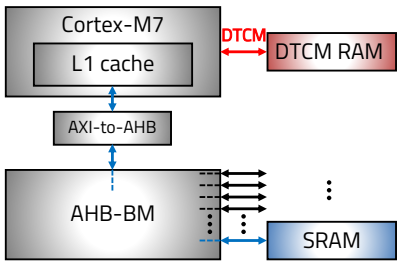

Fig. 1. Memory architecture of STM32F746 MCU

Figure 1 shows the cache memory, SRAM, and DTCM RAM memory architecture of the STM32F746[1] microcontroller unit (MCU). DTCM RAM is tightly coupled with the core through an interface called DTCM, while SRAM is connected through L1-cache and the AHB bus matrix.

Previous research has explored novel cache-aware scratchpad memory (SPM) allocation optimization techniques. However, existing approaches have notable limitations: [3], [4] proposed additional hardware structures for SPM allocation, while [5] and [6] inadequately considered the development environments commonly used in existing MCUs. Work by [7] introduced a contention-aware approach using cache miss counts as allocation metrics, but still requires dedicated hardware components and OS-level modifications, limiting its applicability in typical MCU development workflows. The method proposed in this paper requires no new hardware and instead integrates an additional optimization process into the build system of STM32CubeIDE[2], a widely used development environment for MCUs. Moreover, while a common approach is to model the allocation of frequently accessed data as a 0/1 knapsack problem, allocating data to TCM based solely on access frequency can cause performance degradation in a cache-TCM hybrid architecture. Therefore, our approach

This study was supported by the BK21 FOUR project (4199990113966), the Basic Science Research Program (RS-2018-NR031059, 10%), (RS-2025-24322979, 10%) through the National Research Foundation of Korea (NRF) funded by the Ministry of Education. This work was partly supported by an Institute of Information and communications Technology Planning and Evaluation (IITP) grant funded by the Korean government (MSIT) (No. 2022-0-01170, 20%) and (No. RS-2023-00228970, 30%) and (No. RS-2025-02218227, 20%) and (No. RS-2022-00156389, Innovative Human Resource Development for Local Intellectualization support program, 10%). The EDA tool was supported by the IC Design Education Center (IDEC), Korea.

Mr. Jeon is with Department of Electronic and Electrical Engineering, Kyungpook National University, Daegu, Republic of Korea. Dr. Daejin Park is with School of Electronics Engineering, Kyungpook National University, Republic of Korea. (Correspondence to Daejin Park, email: boltanut@knu.ac.kr)

[1]STMicroelectronics STM32F746 Microcontroller. [Online]. Available: https://www.st.com/en/microcontrollers-microprocessors/stm32f7-series.html

[2]STMicroelectronics Development Tool. [Online]. Available: https://www.st.com/en/microcontrollers-microprocessors/stm32f746.html

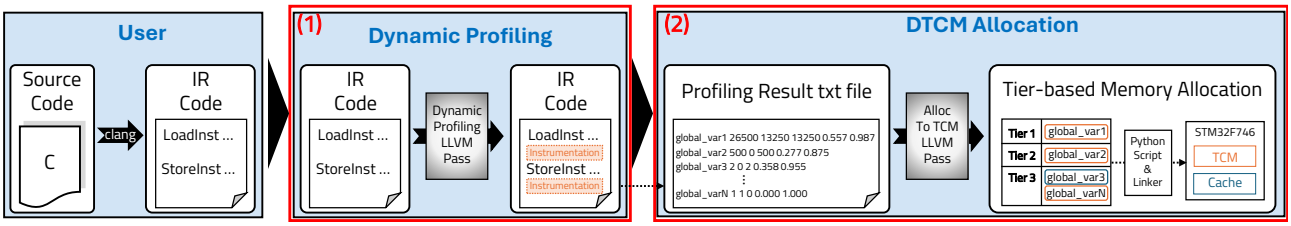

Fig. 2. Overview of the proposed two-stage DTCM allocation framework. (1) The Dynamic Profiling stage instruments the LLVM IR to generate a runtime profile of memory accesses (access counts, locality, etc.). (2) The DTCM Allocation stage uses this profile to classify variables into tiers and applies a hybrid algorithm to allocate them to DTCM or cache.

profiles access type and locality, using the results to allocate data based on a tiered strategy.

II. Background

This section explains our rationale for adopting static compilation and introduces the knapsack problem formulation with dynamic programming, which are used in our approach.

There are two main compilation approaches: static compilation and dynamic compilation. Dynamic compilation performs compilation at runtime, while static compilation completes all compilation before program execution. This paper considers only the static compilation approach for three reasons. First, due to the resource-constrained nature of MCUs, dynamic compilers themselves consume CPU resources, creating significant overhead. Second, dynamic compilation is not suitable for ensuring real-time characteristics of embedded systems [8]. Third, static compilation is more appropriate for achieving consistent performance.

The knapsack problem—selecting a subset of items that maximizes profit under a limited capacity—has often been used in memory allocation research [1], [9]. When the items are indivisible, the problem is called the 0/1 knapsack problem, for which an optimal solution can be obtained via dynamic programming (DP).

III. Implementation

The proposed memory optimization framework is integrated into the STM32CubeIDE build system and, as shown in Figure 2, is implemented as a two-stage process within the LLVM compiler infrastructure. This process consists of a (1) dynamic profiling pass and a (2) data allocation pass. The entire procedure is profile-guided, making data placement decisions based on the program's actual runtime behavior to maximize performance in a hybrid Cache-TCM architecture.

A. Dynamic Profiling via Code Instrumentation

The first stage of the framework gathers empirical data on memory access patterns through dynamic profiling. This is achieved by an LLVM pass that instruments the source code to log memory operations during execution.

1) Instrumentation Mechanism: The pass iterates through the program's LLVM Intermediate Representation (IR). As you can see in Figure 2, for every memory access instruction-specifically LoadInst (read) and StoreInst (write)-that targets a global variable, it inserts a call to a custom C function, record_access_details. This function logs three key pieces: a unique variable ID, an access type flag (read/write), and the actual memory address.

2) Pointer Alias Analysis: To ensure profiling accuracy, the pass must correctly identify the underlying global variable even when it is accessed through a pointer. The framework employs an iterative, flow-insensitive alias analysis to resolve these indirect references. It analyzes store, load, getelementptr, cast, and function call instructions to build a map that traces pointers back to their source global variables. This allows for the correct attribution of all memory accesses.

3) Profiling Output: When the instrumented program is executed on the target hardware, the inserted logging functions generate a detailed runtime record. Upon completion, a summary is produced as a text file, which contains aggregated metrics for each global variable: total access count, read/write counts, a calculated temporal locality score, and a spatial locality score. The temporal and spatial locality score are calculated as follows:

$$S_{\text{spatial}} = \frac{1}{1 + \bar{S}/L} \tag{1}$$

$$S_{\text{temporal}} = \alpha \cdot \frac{1}{1 + \mu/\tau} + (1 - \alpha) \cdot \frac{1}{1 + \text{CV}} \tag{2}$$

$$\text{where} \quad \mu = \overline{\Delta t}, \quad \text{CV} = \frac{\sigma}{\mu}$$

Here, the spatial locality score captures the effect of cache line utilization, where $\bar{S}$ represents the average stride between consecutive accesses and L is the cache line size. And the temporal locality score combines two factors: the average time interval between accesses ($\mu = \overline{\Delta t}$) and access regularity measured by the coefficient of variation (CV = σ/μ). The parameter α balances these two components, while τ serves as a temporal scale parameter to normalize time intervals according to the system characteristics.

B. Multi-Tiered Locality-Aware TCM Allocation

The second stage leverages the profiling data to perform a strategic allocation of global variables to the DTCM RAM. The pass utilizes a multi-tiered classification and a hybrid allocation algorithm to enhance data placement.

1) Variable Classification: Based on the profiling results, all global variables are categorized into several priority tiers. This classification identifies variables based on their potential

performance impact when placed in the DTCM. The primary tiers include:

- Tier 1 (Highest Priority): Read-Dominant & Low-Locality Variables. These variables are ideal for DTCM allocation. Their high read frequency combined with low temporal or spatial locality results in poor cache utilization.
- Tier 2: High-Write & Low-Locality Variables. Numerous write operations can cause pipeline stalls, and data with low locality is more likely to evict other useful data from the cache. Therefore, placing these variables in DTCM RAM can lead to faster operation.
- Tier 3: Low-Access Variables. Variables with very few accesses are not utilized effectively even when placed in cache memory. Since their initial access always causes a cache miss that can degrade performance, placing them in DTCM RAM can further enhance performance.

2) Tiered Allocation Strategy: With variables classified, the pass allocates them to the available DTCM space using a hybrid algorithm tailored to the characteristics of each tier:

- Tier 1 Allocation (Dynamic Programming): A 0/1 Knapsack Dynamic Programming (DP) algorithm is used for the highest-priority variables. The value of each item in the knapsack is a benefit score calculated as follows:

$$\begin{aligned} locality &= (1 - temporal\ locality) \\ &\quad + (1 - spatial\ locality) \\ benefit\ score &= locality \times total\ accesses \end{aligned}$$

  This metric prioritizes variables that are frequently accessed yet have low spatial/temporal locality, making them cache-unfriendly, thereby maximizing the performance gain from DTCM placement.
- Tier 2 & 3 Allocation (Greedy): Any remaining DTCM capacity is filled using a greedy algorithm. Tier 2 variables are sorted by write count in descending order, while Tier 3 variables are sorted by locality in ascending order. This ensures the remaining space is efficiently allocated to the next most impactful candidates.

The finally selected variables are written to a text file, and a Python script uses this file as input to modify the source code, ensuring the defined variables are placed in the DTCM section. During the final linking stage, the linker places these variables into the physical DTCM address space. The entire multi-tiered allocation strategy described in this section is presented in Algorithm 1.

Algorithm 1 Multi-Tiered Locality-Aware DTCM Allocation

Input: $F_{profile}$ (Profiling result file), S_{TCM} (DTCM capacity in bytes)
Output: V_{TCM} (Set of variables allocated to TCM), S_{rem} (Remaining TCM capacity)
$V_{info} \leftarrow$ ParseProfileFile($F_{profile}$) // Read accesses, temporal/spatial localities
$T_1, T_2, T_3 \leftarrow$ ClassifyVariables(V_{info}) // Classify into priority tiers
$S_{rem} \leftarrow S_{TCM}$ // Initialize remaining capacity
$V_{TCM} \leftarrow \emptyset$ // Initialize allocated variables set
// Tier 1: Read-Dominant + Low Locality
$I_1 \leftarrow \emptyset$ // Initialize knapsack items
for each variable $v \in T_1$ **do**
    $locality \leftarrow (1.0 - v.temporal\_locality) + (1.0 - v.spatial\_locality)$
    $benefit\ score \leftarrow locality \times v.totalAccesses$
    Add item $(v, v.size, benefit\ score)$ to I_1
end for
$V_{sel_1} \leftarrow$ KnapsackDP(I_1, S_{rem}) // Run 0/1 Knapsack DP
$V_{TCM} \leftarrow V_{TCM} \cup V_{sel_1}$
$S_{rem} \leftarrow S_{rem} -$ SizeOf(V_{sel_1})
// Tier 2: High Write + Low Locality
Sort T_2 by descending $write\_count$
for each variable $v \in T_2$ **do**
    if $v.size \leq S_{rem}$ **then**
        $V_{TCM} \leftarrow V_{TCM} \cup \{v\}$
        $S_{rem} \leftarrow S_{rem} - v.size$
    end if
end for
// Tier 3: Low Access Count
Sort T_3 by ascending $spatial\_locality$
for each variable $v \in T_3$ **do**
    if $v.size \leq S_{rem}$ **then**
        $V_{TCM} \leftarrow V_{TCM} \cup \{v\}$
        $S_{rem} \leftarrow S_{rem} - v.size$
    end if
end for
return V_{TCM}, S_{rem}

IV. Experimental Setup and Evaluation

This section describes the experimental environment, the LLVM pass used for performance evaluation, and the experimental results.

A. Experimental Setup

The STM32F746 MCU used in this study is equipped with an ARM Cortex-M7 core, which includes 64KB of DTCM RAM. The system operates at 200MHz with the d-cache enabled. The benchmarks used for the experiment, which were ported to the target board, include fft, zip, loop, cjpeg, linpack, and sha from Coremark-pro; qsort and susan from MiBench; and the STREAM benchmark.

B. Experimental Result

Figure 3 shows the change in the clock cycle ratio according to variations in the DTCM RAM size. Although certain benchmarks did not demonstrate substantial execution cycle improvements, the proposed method demonstrates more stable performance gains—avoiding the performance degradation that occurs when cache-friendly data is misallocated to DTCM—compared to a conventional access frequency-based 0/1 knapsack dynamic programming approach, which, as mentioned in the background section, is widely used for memory allocation problems. Furthermore, the MiBench and STREAM benchmarks show significant performance improvements, with

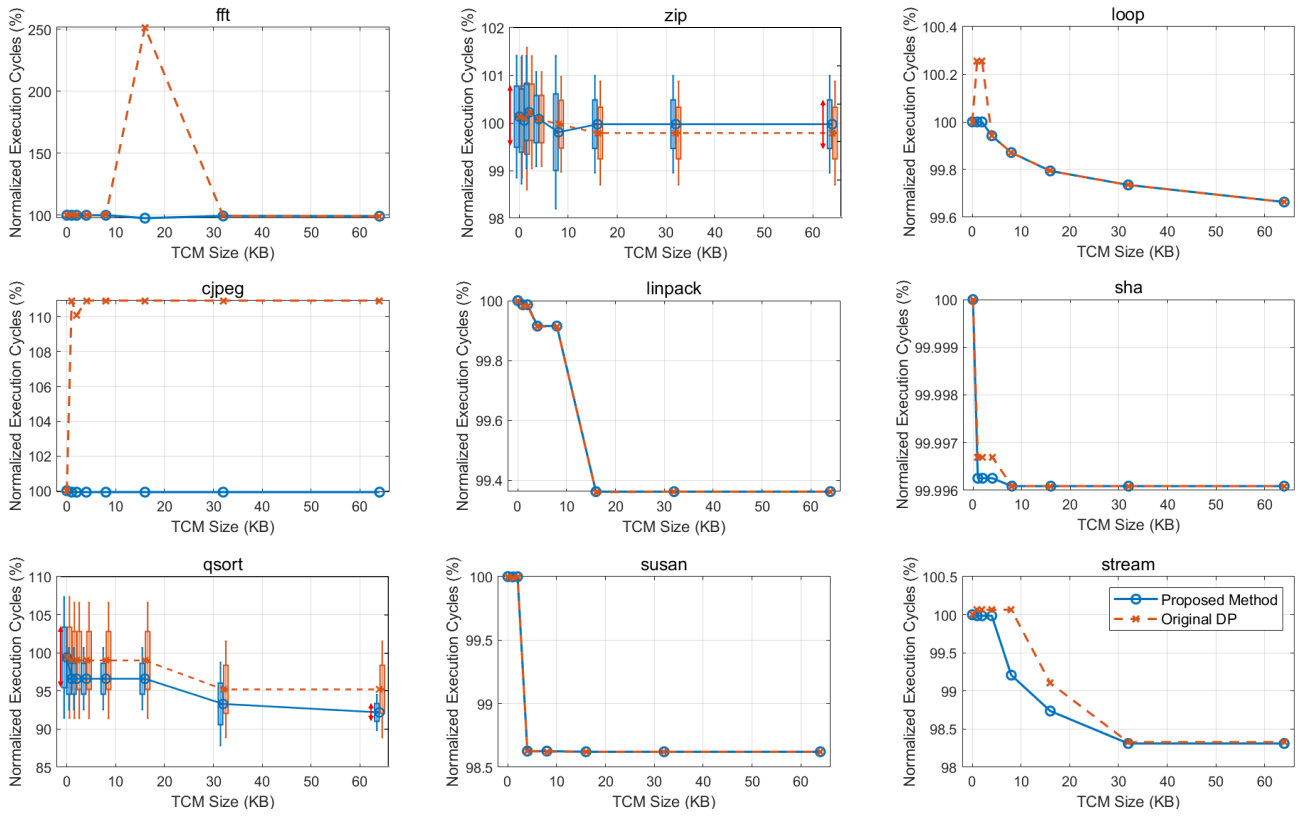

Fig. 3. Experimental results using Coremark-pro, MiBench, and STREAM benchmarks

execution cycle reductions of approximately 8% for qsort, 1.4% for susan, and 1.7% for STREAM, respectively.

Not all experimental results show performance differences between the proposed method and access frequency-based DP, as these differences mainly stem from the characteristics of global variables in each benchmark. When benchmarks contain many read-dominant, low-locality, small objects, the proposed method shows clearer advantages; when dominated by large arrays with moderate locality, both methods perform similarly.

ZIP and qsort are algorithms where external data causes changes in internal software execution paths, leading to execution time variability that is visualized through candlestick plots. The red arrows highlight the contrast between cache-only scenarios (high variability) and 64KB TCM configurations (reduced variability), confirming improved time predictability.

V. Conclusion

This research addressed the effective allocation of hybrid memory architectures, balancing the trade-offs between automated cache management and explicit TCM control. We proposed a two-phase approach: profiling using an LLVM pass to collect access metadata, followed by tier-based DTCM allocation based on these metrics.

The experimental results demonstrate that the proposed method achieves lower clock cycle consumption, more stable performance compared to the original DP approach, and enhanced time-predictability over cache-only configurations.

References

[1] R. Banakar, S. Steinke, B.-S. Lee, M. Balakrishnan, and P. Marwedel, "Scratchpad memory: a design alternative for cache on-chip memory in embedded systems," in *Proceedings of the Tenth International Symposium on Hardware/Software Codesign. CODES 2002 (IEEE Cat. No.02TH8627)*, 2002, pp. 73–78.

[2] M. Verma, L. Wehmeyer, and P. Marweclel, "Dynamic overlay of scratchpad memory for energy minimization," in *International Conference on Hardware/Software Codesign and System Synthesis, 2004. CODES + ISSS 2004.*, 2004, pp. 104–109.

[3] L. Wu and W. Zhang, "Cache-aware spm allocation algorithms for hybrid spm-cache architectures," in *Sixteenth International Symposium on Quality Electronic Design*, 2015, pp. 123–129.

[4] N. A. Siddique, A.-H. A. Badawy, J. Cook, and D. Resnick, "Lmstr: Local memory store the case for hardware controlled scratchpad memory for general purpose processors," in *2016 IEEE 35th International Performance Computing and Communications Conference (IPCCC)*, 2016, pp. 1–8.

[5] H. Wen and W. Zhang, "Reducing cache leakage energy for hybrid spm-cache architectures," in *2014 International Conference on Compilers, Architecture and Synthesis for Embedded Systems (CASES)*, 2014, pp. 1–9.

[6] H. Wang, Y. Zhang, C. Mei, and M. Ling, "Energy-oriented dynamic spm allocation based on time-slotted cache conflict graph," in *2010 Design, Automation & Test in Europe Conference & Exhibition (DATE 2010)*, 2010, pp. 598–601.

[7] D.-W. Chang, I.-C. Lin, Y.-S. Chien, C.-L. Lin, A. W.-Y. Su, and C.-P. Young, "Casa: Contention-aware scratchpad memory allocation for online hybrid on-chip memory management," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 33, no. 12, pp. 1806–1817, 2014.

[8] M. Schoeberl, "A java processor architecture for embedded real-time systems," *Journal of Systems Architecture*, vol. 54, no. 1, pp. 265–286, 2008. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1383762107000963

[9] V. Suhendra, T. Mitra, A. Roychoudhury, and T. Chen, "Wcet centric data allocation to scratchpad memory," in *26th IEEE International Real-Time Systems Symposium (RTSS'05)*, 2005, pp. 10 pp.–232.